Exploring Architectural Smells Detection Through LLMs

Claudio Tessa, Matteo Bochicchio(✉), and Francesca Arcelli Fontana

University of Milano-Bicocca, Milan, Italy
c.tessa@campus.unimib.it, {matteo.bochicchio,francesca.arcelli}@unimib.it

Abstract. Architectural smells (AS) are design flaws in software systems that, if left unaddressed, can negatively impact maintainability and system evolution. This preliminary study investigates the use of Large Language Models (LLM) for detecting and explaining an AS called Hub-like Dependency (HL), a critical smell type characterized by components with numerous incoming and outgoing dependencies. The research leverages Google's Gemini 1.5 Pro, comparing its performance to Arcan, a specialized AS detection tool. The study analyzes 135 AS across 39 open-source Java projects, including 100 Hub-like Dependency smells with varying severity levels and 35 non-Hub-like Dependency smells. Results show that the LLM achieves 100% recall but varying precision, with more detailed prompts improving detection performance from 64% to 82% for lower-severity smells. However, the model's ability to generate human-understandable explanations remains limited, with only 49% of the generated explanations rated as satisfactory. These findings highlight both the potential and current limitations of one specific LLM in HL smell detection, suggesting the importance of prompt design in enhancing its capabilities.

Keywords: Architectural Smells detection · LLM · Hub-like Dependency smell · Architectural Smells explainability

1 Introduction

Architectural Smells (AS) are patterns or structures within software systems that violate design principles, and therefore negatively impact design quality. If not addressed, these smells can accumulate over time, leading to poor maintainability [6] and technical debt [16]. For this reason, early detection and resolution of AS is crucial. In this study, we focus our attention on one frequent and critical AS, the Hub-like Dependency (HL) smell. This smell arises when an abstraction has a large number of both incoming and outgoing dependencies with other abstractions, making the dependency graph look like a hub [16]. The more traditional approaches to detecting AS rely usually on rule-based systems. However, these methods can miss architectural issues that require a deeper understanding of the system's design [5]. This is where Large Language Models (LLMs) could

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
V. Andrikopoulos et al. (Eds.): ECSA 2025, LNCS 15929, pp. 90–98, 2026.
https://doi.org/10.1007/978-3-032-02138-0_6

offer a promising solution. LLMs can process and analyze large amounts of code and have already shown promising potential in supporting code maintenance activities, such as detecting test smells [11], code smells [19], and vulnerabilities [12]. However, to the best of our knowledge, no study to date has specifically focused on the detection of AS using LLMs. This preliminary study will analyze how HL smells can be detected with the use of LLMs, in particular Google's Gemini 1.5 Pro[1], and compare its performance to Arcan, a specialized tool for AS detection [3]. Furthermore, we will examine whether LLMs could be useful in explaining the identified smells, i.e., providing insights into the underlying causes of the AS and possible refactoring suggestions. All the data is publicly available on Zenodo [17].

2 Related Work

The detection of AS in software systems has been an area of active research within the field of software engineering, as outlined in the works below. Traditional approaches to detecting these smells have largely relied on static analysis tools. Recently, the advent of Machine Learning (ML) and LLMs has opened new avenues for analyzing and understanding such smells. Tools like Arcan[2] and SonarQube[3] analyze source code to detect architectural or code smells, relying on manually curated rules and metrics like coupling, cohesion, and complexity. These methods are highly effective in detecting well-known smells, however, they often struggle with contextual nuances and cannot easily adapt to modern software architectures, as noted by Ernst et al. [5]. To address the limitations of static tools, ML methods have emerged. Supervised ML models have achieved impressive results, with Arcelli et al. [2] reporting up to 99% accuracy in detecting code smells like Data Class. Additionally, unsupervised methods such as Self-Organizing Maps have been applied to anomaly detection, achieving 90%-94% accuracy without requiring labeled data [7]. Despite these advances, ML-based methods have largely focused on code smells rather than AS. More recently, LLMs have also been evaluated for code smell detection, with ChatGPT reaching up to 70% accuracy on test smells [11,15], especially when using detailed prompts. Moreover, LLMs have been applied to technical debt explanations [12], though their performance on AS remains underexplored and is an ongoing area of investigation.

3 Study Design

Our goal is to evaluate LLMs (in particular, Google's Gemini 1.5 Pro) performance in the detection and explainability of HL smells. We elaborated the following research questions (RQ):

[1] See https://deepmind.google/technologies/gemini/.
[2] See https://arcan.tech.
[3] See https://www.sonarsource.com/products/sonarqube/.

RQ1: *To what extent can LLMs detect HL smells in Java code?*

RQ1.1: *How does a HL smell's severity affect the LLM's ability to detect it?*

The answers to these questions can help us understand some LLM limitations (in particular, of Gemini 1.5 Pro) and whether there are biases in identifying smells with varying degrees of criticality (severity). We compare the LLM detection performance with that of the Arcan tool.

RQ2: *How effective are LLMs in providing human-understandable explanations for the identified AS?*

Providing meaningful explanations can help developers understand the underlying causes and potential consequences of having AS in their codebase. Moreover, these explanations could also be useful for refactoring the smell.

Architectural Smells. We decided to use the tool Arcan (the first version of the tool has been developed at our lab [3]) able to detect several AS and provide a graph representation of each smell, useful to understand and remove the smell[4]. Moreover, the tool has been used in several previous studies (such as in [1,8,14]). In this paper, we focus our attention on the Hub-like Dependency smell (HL), which arises when a component has a high number of incoming and outgoing dependencies, becoming a central point of control. We decided to focus our attention on this smell, since an HL is generally bad for the project maintainability, as changes to the central "hub" component can have a cascading effect on all the other modules that depend on it or that it depends on [16]. An AS can arise at both the package level and at the class level. In this preliminary study, we focus only on class-level smells, as package-level smells often contain too much code to be analyzed by the considered LLM. HL smells are classified by Arcan with varying levels of severity (1–10), which indicate the complexity, criticality of the smell. For our study, we selected only the smells with severity levels 6 and 7 (the most common ones), in order to verify whether the model is capable of handling sufficiently complex and non-trivial smells. An example of a HL dependency graph of severity 7 is provided in the replication package [17].

Analyzed Projects. For our evaluation, we chose 39 open-source projects from GitHub. according to a set of prerequisites: (1) the project must have a code base written for at least 30% in the Java programming language; (2) be actively maintained, with the latest updates being no more than 2 years ago; (3) have at least 100 stars on GitHub, indicating that it has a certain level of interest within the developer community; (4) be analyzable by Arcan; (5) have at least one severity-6 or severity-7 HL smell, as classified by Arcan. According to these constraints, we were able to analyze a total of 100 HL smell instances. We then individuated from these projects 35 more AS that are not HL smells, considering the same requirements as above, except the last one. These instances will serve to evaluate false positives.

[4] https://www.arcan.tech/.

Selection of the LLM and Prompts. In this study, we used Google Gemini 1.5 Pro because it has a declared context window of one million tokens, which is sufficiently large for our work. This extensive context length made it feasible to analyze HL smells, which often consist of tens of thousands of Lines of Code (LOC) and hundreds of thousands of tokens. Before this decision, we evaluated other alternative LLMs, both open-source and proprietary (see [17]).

Different *prompting techniques* have been studied to try and increase LLMs performance on specific tasks. The ones we used to construct our prompts are Zero-shot [4], Role-Play [9], and Chain-Of-Thought (COT) prompting [18]. We identified Prompts 1 and 2 to evaluate how the detection performance differs from each prompt style, and one Prompt 3 to assess the model's reasoning capabilities in explaining the AS using the COT prompting technique. For the details of the prompts see the replication package [17].

Prompt 1. No definition was given on what an HL smell is, but it was given all the Java code involved in the smell.

Prompt 2. It was given the definition of the HL smell [16] and the Java code involved in the smell.

Prompt 3. It was given all the Java code involved in the smell and asked to explain it. We evaluated the model's capabilities in explaining HL smells and give some suggestions on refactoring. We used COT prompting to give the model an example of what we were expecting and the thought process behind our answer.

Each AS was given to the model with every prompt, each in a separate new chat. This ensured that the previous context of the chat did not affect the model's performance. For each smell, all the comments from the Java code were removed. This not only reduced the token count (and thus the complexity of the prompt) but it also ensured that the model did not receive any hints from self-admitted technical debt in the comments.

Evaluation Methods. Given the probabilistic nature of LLMs, the output may vary considerably across interactions, even when provided with identical input. To reduce the impact of stochastic variability and ensure the robustness of our evaluation, each unsuccessful prompt was tested *up to three times*, with each attempt performed in a separate, newly initialized chat session. A prompt was considered successful if the model produced the correct response within these three attempts [11]. Once a correct response was obtained, no further trials were conducted. To evaluate the performance of the model in detecting HL smells, we used three metrics often used in ML and information retrieval: *Precision*, *Recall* and *F-Measure* [13]. While to evaluate the performance of the model in the explainability of the given AS, we consider the following three criteria, proposed by Mao et al. [12]: 1 - *Correctness*: the explanation must be factually correct and describe how the smell came to be there in the first place, and which insights the developers can gain from the model's response; 2 - *Clarity*: the explanation must be given in a concise way and be understandable by a software developer; 3 - *Refactoring suggestions*: the explanation must contain a high-level description of how to remove the HL smell, or how to reduce its impact.

We evaluated the responses manually and rated them with the predefined rating scale and criteria, although the subjectivity of this evaluation poses significant challenges to its validity. The evaluations have been done by the three authors expert of this context, with a particularly deep attention of the first author. Based on the aforementioned judging criteria, the rating scale is the following: a score is given and it can be either **1** - indicating satisfaction or **0** - indicating non-satisfaction. An arithmetic average of the results was then calculated to obtain a representative value of the overall performance. Finally, we divided the results into two groups, based on the severity level of the smells, severity-6 and severity-7, to evaluate if there is a correlation between the severity level of the smells and the model's performance.

4 Results

Detection Performance to Answer RQ1 and RQ1.1. Table 1 shows the detection results of the smells divided by severity level and prompt used. To address **RQ1** we designed experiments employing two different prompts: Prompt 1 with a basic level of information, and Prompt 2 with a more detailed definition of the HL smell. Although both prompts achieve perfect recall (1.00) across severity levels, meaning they successfully identify all true instances, there is a notable difference in precision. Prompt 2 outperforms Prompt 1 in precision, indicating that it reduced the number of false positives. In answering **RQ1.1** we observed that the detection performance deteriorates as the severity of the smell increases. This is particularly evident in Severity-7 precision score, where Prompt 1 achieves only 0.50 precision, compared to 0.71 in Prompt 2. A more comprehensive overview of these results can be obtained by looking at the F-measure score, which combines both precision and recall. For both Prompt 1 and Prompt 2, the F-measure drops substantially when going from Severity-6 to Severity-7. For the model, it is more difficult to understand more complex smells or smells with more LOC (see Table 1). Overall, while some of these results may seem promising for LLMs, especially for Prompt 2, it is important to note that the dataset is unbalanced. As mentioned before, the non-Hub-like smells only comprise about 26% of our dataset, which may inflate recall.

Table 1. Detection results on Prompt 1 and Prompt 2 and LOC statistics.

Severity	Prompt	Precision	Recall	F-measure	Avg LOC	Median LOC
6	1	0.64	1.00	0.78	4 163	3 305
7	1	0.50	1.00	0.67	9 113	4 177
6	2	0.82	1.00	0.90	4 163	3 305
7	2	0.71	1.00	0.83	9 113	4 177

Explainability Performance to Answer RQ2. We evaluated the answers of the model giving a score of either 0 (not satisfactory) or 1 (satisfactory) [12], and then calculated the percentage of satisfactory answers[5]. A Likert scale will be used in future works. The percentage of satisfying explanations is **0.53%** for *severity-6* smells and **0.41%** for *severity-7* smells, resulting in an overall rate of 0.49% satisfactory responses. These results suggest that while the model shows promise in generating understandable explanations for AS, its performance varies according to complexity. Higher severity smells may involve more intricate dependencies (and, therefore, a more complex dependency graph) that the model is not properly able to understand.

According to RQ2 and the answers received from Prompt 3, in Table 2 we summarize the most frequent suggested refactorings (for the complete table, please refer to [17]). These suggestions align with best practices for reducing coupling and improving modularity in software architecture. Among the design patterns recommended by the model, the Façade pattern (56 occurrences) and the Observer pattern (13 occurrences) were suggested more frequently, likely because they help encapsulate dependencies and promote better separation of concerns. Other patterns such as Strategy, Mediator, and Factory were also occasionally proposed, but with significantly lower frequency. These findings highlight that the LLM, while capable of suggesting various refactoring techniques, tends to rely on a core set of well-known remedies, particularly those aimed at reducing dependency complexity at the class level rather than restructuring the system's architecture at a higher level.

Table 2. Most suggested refactorings (from Prompt 3)

Proposed remedy	Number of times proposed
Introduce interfaces or abstract classes	46
Decompose classes	68
Dependency injection	71
Strategy pattern	10
Observer pattern	13
Façade pattern	56
Mediator pattern	11

5 Threats to Validity

Several factors may affect the validity of this study. Evaluation metrics for AS detection (precision, recall, F-measure) depend on dataset size and balance; a

[5] See the rep. package [17] for examples of satisfactory answers and prompt responses.

small, skewed dataset may inflate recall. However, the data includes AS of varying severities, sizes, and projects to reflect real-world diversity. Subjective rating of LLM-generated explanations introduces potential bias, despite predefined criteria. Standardized evaluation could yield different results. LLM stochasticity also adds variability; prompts were run three times to reduce this, though some randomness remains. The study is limited to HL smells at class level detected using the Arcan tool in open-source Java projects. Therefore, the findings may not be generalizable to other types of AS, programming languages, industrial systems, or AS detection tools. Additionally, the focus on severity-6 and -7 smells may further reduce the representativeness of the results. The LLM was tested with default settings to assess general-purpose performance. Larger models like Qwen2.5-Coder and Code LLaMA were excluded due to hardware limits and insufficient context windows for AS up to 700k tokens.

6 Conclusions and Future Work

The findings of this study highlight both the opportunities and limitations of integrating LLMs into software maintenance workflows. According to **RQ1** and **RQ1.1**, a key insight is the significant influence of prompt design on detection performance. Providing contextual information, such as a formal definition of HL smells, improved precision by 1520%, while recall remained stable. Additionally, the model's performance varied with the severity of AS: higher-severity smells (severity-7) posed greater challenges, especially in terms of precision and explanation quality. COT prompting improved reasoning slightly but did not fully mitigate performance drops on more complex smells. According to **RQ2**, the analysis of the explainability results shows that while the LLM is capable of producing understandable explanations for simpler HL smells, its effectiveness decreases notably as complexity increases. On average, only 49% of the explanations were rated satisfactory, with lower performance observed on more severe smells. Nevertheless, when successful, the model consistently suggests well-known refactoring strategies that align with best practices for reducing coupling and improving modularity. This indicates that the LLM can effectively support developers by reinforcing established design principles, particularly in less complex scenarios or when supplemented by clear, domain-specific prompts. Different improvements of this study can be considered: model fine-tuning on domain-specific datasets and the adoption of emerging techniques like Retrieval Augmented Generation (RAG) [10], which could enhance the model's capacity to provide accurate explanations by leveraging external knowledge sources, consider different types of AS, other programming languages, and additional LLM architectures. Investigating alternative prompting strategies, such as few-shot prompting, could also yield better performance. Finally, a more extended evaluation framework involving multiple experts or real-world validation in collaboration with developers could provide more reliable insights into the practical utility of LLM-generated explanations.

Data Availability. All analyses conducted with Arcan, along with all the code involved in the smells, the prompts used, and the obtained results, are publicly available in the replication package on Zenodo [17].

References

1. Arcelli, F.F., Locatelli, F., Pigazzini, I., Mereghetti, P.: An architectural smell evaluation in an industrial context. In: ICSEA 2020, pp. 68–74 (2020)
2. Arcelli Fontana, F., Mäntylä, M.V., Zanoni, M., Marino, A.: Comparing and experimenting machine learning techniques for code smell detection. Empir. Softw. Eng. **21**, 1143–1191 (2016)
3. Arcelli Fontana, F., Pigazzini, I., Roveda, R., Zanoni, M.: Automatic detection of instability architectural smells. In: ICSME 2016, pp. 433–437. IEEE Computer Society (2016). https://doi.org/10.1109/ICSME.2016.33
4. Dang, H., Mecke, L., Lehmann, F., Goller, S., Buschek, D.: How to prompt? Opportunities and challenges of zero- and few-shot learning for human-AI interaction in creative applications of generative models. In: ACM CHI Conference - Workshops (2022). https://www.hciai.uni-bayreuth.de/pool/docs/workshops%5fCHI22.pdf
5. Ernst, N.A., Bellomo, S., Ozkaya, I., Nord, R.L.: What to fix? Distinguishing between design and non-design rules in automated tools. In: 2017 ICSA, pp. 165–168 (2017). https://doi.org/10.1109/ICSA.2017.25
6. Garcia, J., Popescu, D., Edwards, G., Medvidovic, N.: Identifying architectural bad smells. In: 2009 13th CSMR, pp. 255–258 (2009). https://doi.org/10.1109/CSMR.2009.59
7. Gupta, R., Kumar, N., Kumar, S., Kumar Seth, J.: Unsupervised machine learning for effective code smell detection: a novel method. J. Commun. Softw. Syst. **20**(4), 307–316 (2024)
8. Herold, S.: An initial study on the association between architectural smells and degradation. In: Jansen, A., Malavolta, I., Muccini, H., Ozkaya, I., Zimmermann, O. (eds.) ECSA 2020. LNCS, vol. 12292, pp. 193–201. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-58923-3_13
9. Kong, A., et al.: Better zero-shot reasoning with role-play prompting. In: NAACL-HLT (2024)
10. Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. Adv. Neural. Inf. Process. Syst. **33**, 9459–9474 (2020)
11. Lucas, K., Gheyi, R., Soares, E., Ribeiro, M., Machado, I.: Evaluating large language models in detecting test smells. arXiv preprint arXiv:2407.19261 (2024)
12. Mao, Q., Li, Z., Hu, X., Liu, K., Xia, X., Sun, J.: Towards effectively detecting and explaining vulnerabilities using large language models. arXiv preprint arXiv:2406.09701 (2024)
13. Naidu, G., Zuva, T., Sibanda, E.M.: A review of evaluation metrics in machine learning algorithms. In: Silhavy, R., Silhavy, P. (eds.) CSOC 2023. LNNS, vol. 724, pp. 15–25. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-35314-7_2
14. Sas, D., Avgeriou, P., Arcelli Fontana, F.: Investigating instability architectural smells evolution: an exploratory case study. In: IEEE ICSME 2029, pp. 557–567 (2019). https://doi.org/10.1109/ICSME.2019.00090
15. Silva, L.L., Silva, J.R.d., Montandon, J.E., Andrade, M., Valente, M.T.: Detecting code smells using chatGPT: initial insights. In: Proceedings of the 18th ESEIW (2024)

16. Suryanarayana, G., Samarthyam, G., Sharma, T.: Refactoring for Software Design Smells: Managing Technical Debt. Morgan Kaufmann (2014)
17. Tessa, C., Bochicchio, M., Arcelli Fontana, F.: Dataset: exploring architectural smells detection through LLMs (2025). https://doi.org/10.5281/zenodo.15488631
18. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q.V., Zhou, D., et al.: Chain-of-thought prompting elicits reasoning in large language models. Adv. Neural. Inf. Process. Syst. **35**, 24824–24837 (2022)
19. Wu, D., et al.: iSMELL: assembling LLMs with expert toolsets for code smell detection and refactoring. In: Proceedings of the 39th ASE, pp. 1345–1357 (2024)